

FC Source Changes — UAVXArmQ

Overview

This document describes the major architectural changes in the UAVXArmQ firmware. These are not minor tweaks — they restructure how the FC thinks about attitude, altitude, and safety.

1. Quaternion Attitude Representation

What

UAVXArmQ replaces the legacy Euler-angle / DCM attitude representation with a full quaternion formulation. The core attitude filter, control mixing, and axis transformation all operate on **f32** quaternions rather than roll/pitch/yaw angles internally.

Specific changes:

- **Attitude estimator:** The complementary filter (Madgwick) outputs a unit quaternion instead of Euler angles. The optional Kalman filter path also uses quaternion state.
- **Control pipeline:** Rate and angle controllers receive quaternion error directly, avoiding gimbal-lock-prone Euler decomposition. The `Quat2Euler()` conversion is only called for telemetry output and user-facing displays.
- **QUAT_GAIN parameter** (tag 75): New tuning parameter that scales quaternion error into the angle controller, replacing the per-axis angle Kp interpolation.

Why

Euler angles suffer from gimbal lock at $\pm 90^\circ$ pitch and require trigonometric conversions at every control iteration. Quaternions provide:

- No singularities — valid for all orientations including inverted flight
- Linear interpolation (SLERP) for smooth setpoint transitions
- Single representation for both estimation and control — no back-and-forth conversion
- Lower computational cost: quaternion multiply (4 mul + 3 add) vs. Euler angle trig functions

The switch is transparent to the user: the GCS still displays roll/pitch/yaw, but the FC thinks in quaternions internally.

2. Altitude Hold — Removal of Cascaded Rate Control

What

The altitude controller was simplified from a two-stage cascade (position PID $\rightarrow$ velocity PID $\rightarrow$ throttle) to a single-stage position PID that feeds directly into the throttle mixer.

Removed parameters:

- `ALT_VEL_KP` $\rightarrow$ UNUSED (tag 29)
- `ALT_VEL_KI` $\rightarrow$ UNUSED (tag 101)
- `ALT_VEL_INT_LIMIT` $\rightarrow$ UNUSED (tag 66)

The remaining `ALT_POS_KP` (tag 6), `ALT_POS_KI` (tag 1), and `ALT_POS_INT_LIMIT` (tag 99) now directly produce a throttle correction that is added to the cruise throttle feedforward.

Why

The cascaded velocity loop was designed for a barometer-driven altitude controller with significant measurement lag. With the introduction of the GPS-plus-barometer altitude estimator and the cruise throttle feedforward (see §4), the intermediate velocity loop was redundant:

- The feedforward term (`pCruiseThrottle`) already provides the bulk hover thrust
- The position PID only needs to correct for small disturbances (wind, battery voltage sag)
- Removing the velocity loop eliminates a derivative path that amplified barometer noise
- Simplifies tuning: only one Kp/Ki pair instead of two

The removed parameter slots are preserved in the parameter table (as `UNUSED`) to maintain index alignment with the original 128-parameter layout.

3. Dive Recovery

What

A new `DiveRecover()` function in `control.c` detects rapid uncommanded altitude loss and automatically applies full throttle with a leveling attitude override.

- **Trigger:** Altitude loss exceeds `DIVE_RECOVER_ALT` meters (tag 116) within a 500 ms window, combined with throttle below the cruise threshold.
- **Action:** Forces throttle to maximum, levels the attitude via quaternion SLERP to upright, and holds for 1.5 s before releasing control back to the pilot/nav system.
- **Indication:** Sets a `DIVE_RECOVERY_ACTIVE` flag visible in the tuning telemetry packet (tag 57).

Implementation is in `DoControl()` — the dive check runs every control cycle and preempts both the angle controller and the navigation system while active.

Why

Multirotors descending rapidly (e.g., after a free-fall from a flip, or descending into own propwash) can enter a vortex-ring state (VRS) where the props produce little thrust. The pilot may not recognize the sink rate in time. Automated dive recovery:

- Catches the condition before VRS fully develops
 - Applies the correct recovery consistently — full throttle, level attitude
 - Does not require pilot action or specific switch position
 - Automatically disengages when the aircraft has recovered altitude, returning control to the pilot
-

4. Unified Float Parameter System

What

All 128 parameters are stored as native `float32` in `Config.ParamData[i].f`. Legacy `uint8` storage, `LEGACY_TAGS`, `PARAM_SCALES`, and the `GCS_legacy_mode` conversion tables are removed.

Wire Protocol

- Tag 17: Individual write — `index(1) + float32(4)`
- Tag 71: Bulk readback — `base_idx(1) + count(1) + float32×count`
- Display: `display = fc_float × PARAM_DISPLAY_MULT[idx]`
- Range: `PARAM_LIMITS[idx][0] × mult` to `PARAM_LIMITS[idx][1] × mult`

Why

The mixed uint8/float32 system required the GCS to track which format each parameter used, doubling the conversion complexity and introducing a source of bugs. Storing everything as float32:

- Single wire format — every read/write is 4 bytes, IEEE-754
 - Uniform dynamic range for all parameters ($\pm 1e38$ vs. 0–255)
 - Simplifies black-box logging (all params same size and type)
 - Enables fractional values for parameters that were previously quantized to integer steps
-

5. Cruise Throttle Feedforward v2

What

`TrackCruiseThrottle()` now tracks `AltHoldThrComp` directly instead of `DesiredThrottle + AltHoldThrComp - pCruiseThrFF`. Tracking rate increased from 0.5 %/s to 2 %/s. The feedforward term is persisted on disarm and no longer overridden by `ApplyParameters()`.

Why

The original formulation was sensitive to rate-of-climb noise and converged too slowly. Key improvements:

- **Cleaner signal:** `AltHoldThrComp` is the feedback PI correction — it directly represents the power error from hover. Tracking it isolates the hover estimate from throttle stick movement and ROC noise.
- **4× faster convergence:** 2 %/s vs. 0.5 %/s — a 15 % error corrects in 7.5 s
- **No ROC guard:** `ThrottleMoving` is the only gate; the old ROC-based freeze is removed
- **Persistence:** Learned value survives power cycles via write-on-disarm
- **Integrators zeroed on engagement:** Prevents stale `AltHoldThrComp` from offsetting the feedforward at hold entry

Related

- `pMaxAltHoldThrComp` default increased from 0.15 to 0.25 (25 % max feedback correction)
-

6. Arming and NavQual System

What

Channel 6 (`NavQualificationRC`) provides dual-purpose arming qualification and progressive mode enable:

Method	Arm	AltHold	WPNav
Switch	Physical switch (Ch6)	Always on	Always on
Tx (pot)	Ch6 > 10 %	> 40 %	> 60 %

Why

Consolidates arming logic into a single channel, freeing remaining channels. The potentiometer method allows pilots to arm and progressively enable modes without a dedicated arm switch — useful on transmitters with limited switches.

All `F.AltControlEnabled` = assignments live in `rc.c MapRC()`; no other file writes this flag.

7. Failsafe

What

Two independent paths to the same behaviour in `MapRC()`:

1. **SBus failsafe flag**: `sbusDecode()` sets `RCFailsafe` → `MapRC()` substitutes `FS[c].Raw` presets
2. **Timeout failsafe**: `GenerateFailsafePacket()` loads presets into `RCInp[]` and sets `RCFailsafe`

Presets: `NavQual` = 1650 (65 %), `NavMode` = 2000 (RTH), `sticks` = 1500 (neutral), `throttle` = 1050 (idle).

Override

At the end of the `NavQual` block, `RCFailsafe || RCSignalLost` forces `AngleMode` + `AltHold` regardless of switch positions. `PassThru` overrides all failsafe behaviour (direct sticks, no nav, for fixed-wing manual recovery).

Future

A configurable failsafe action parameter is planned (enum in FC: `FS_Action`: `RTH` / `Land` / `MotorsOff`; combo box in GCS).

8. Tuning Telemetry Packet (Tag 57)

What

`SendTuningPacket()` emits a compact 22-byte telemetry frame every cycle (~50 Hz):

`flags(1) + throttle(1) + 3 × [desired_rate(2) + actual_rate(2) + axis_out(2)]`

Replaces the legacy `SendAttitudeControlPacket()` (tag 68) which cycled one axis per invocation at ~2.5 Hz.

Why

The legacy packet was gated behind `pBBLogType`, fired only one axis per call, and used mixed precision (`int16 × 1000` for rate, `int8 × 200` for output). The new packet:

- All three axes every cycle, uniform 16-bit precision
- Includes throttle and mode flags
- Eliminates axis-cycling complexity in `control.c`
- Provides a single predictable stream for real-time GCS tuning

9. Navigation Enhancements

What

Several navigation parameters were added or modified:

- **NAV_CROSS_TRACK_KP** (tag 48): Proportional gain for cross-track error correction during waypoint navigation. Previously the cross-track correction was a fixed constant.
- **NAV_PROX_ALT_M** (tag 106) and **NAV_PROX_RADIUS_M** (tag 107): Proximity detection — the FC reduces speed when obstacles or the ground are within the proximity window.
- **NAV_FENCE_RADIUS_M** (tag 115): Geofence radius. The FC initiates RTH if the aircraft exceeds this distance from home.

- **NAV_HEADING_TURNOUT** (tag 78): Turn coordination parameter — controls how aggressively the heading controller leads turns during waypoint following.

Why

These parameters provide finer-grained control over autonomous navigation behaviour, replacing hard-coded constants with tunable values accessible from the GCS parameter editor.